CRITICAL ANALYSIS

v3.0

CryptoQuant Terminal Strategy Decision Engine Design

Deep architecture review from Quant Developer, Portfolio Manager, and Risk Manager perspectives. 30 key questions answered. Strategy Decision Engine designed. Prioritized 6-sprint roadmap toward semi-autonomous operation.

DATE 2026-06-04

VERSION 3.0 — Comprehensive Analysis

SCOPE Architecture, Decision Engine, Capital Allocation, Risk

ROLES Quant Dev / Portfolio Mgr / Risk Mgr

Table of Contents

Part I: Executive Summary

CryptoQuant Terminal is an ambitious quantitative crypto trading platform built on Next.js 16 + Prisma 7 + SQLite. It contains 17+ functional modules, 45+ API endpoints, and 16 capital allocation methods. However, the system has a fundamental architectural flaw: it produces information but not decisions. A user sitting in front of the dashboard sees signals, scores, Monte Carlo metrics, Walk-Forward Efficiency, operability grades, lifecycle phases, and system recommendations. But the question that truly matters — "Should I allocate capital to this strategy NOW, and how much?" — has no direct answer. The user must mentally synthesize the output of 5-6 modules to reach a conclusion.

This is analogous to having a team of analysts where each presents their report independently but nobody makes the final decision. The modules operate as islands with weak interconnection, and the most critical piece — a Strategy Decision Engine that synthesizes all module outputs into actionable portfolio-level decisions — does not exist yet. This document provides a comprehensive critical analysis from three professional perspectives (Quant Developer, Portfolio Manager, Risk Manager), answers 30 key questions about architecture and direction, designs the Strategy Decision Engine, and presents a prioritized roadmap toward semi-autonomous and autonomous operation.

> **Key Finding:** The project has exceptional individual modules (Monte Carlo, Walk-Forward, Capital Allocation with 16 methods, Evolution Engine) but zero integration between them. The Strategy Decision Engine is the missing keystone that transforms information into action. Without it, the platform is a sophisticated analytics dashboard, not a trading system.

Current Architecture State

Module	Lines	Status	Connected?	Critical Issue
Decision Engine	583	Token-level only	Partial	Not strategy-level
Capital Allocation	1,097	16 methods complete	No	No automated consumer
Monte Carlo	727	Production quality	No	No Block Bootstrap
Walk-Forward	655	Complete WFA	No	No parameter optimization
Evolution Engine	1,063	GA with mutation	No	Math.random() non-reproducible
Paper Trading	1,100	Full simulation	Partial	Simple position sizing
Brain Orchestrator	1,040	11-phase pipeline	Partial	Output not consumed by SDE
Strategy State Mgr	N/A	6 states tracked	No	State does not control execution
Backtest Engine	1,200	Direction-aware	No	Simple position sizing

Risk Module	11 files	Operability + alerts	Partial	No VaR, no kill switches

Table 1: Current architecture state of all major modules

Integration Gap Map

The following diagram illustrates the fundamental problem: modules produce outputs but those outputs are not consumed by any decision-making layer. Each arrow represents a data flow that should exist but currently does not. The Strategy Decision Engine (shown in red) is the missing keystone that must be built to unify the system.

Source Module	Output	Target Consumer	Currently Connected?
Monte Carlo	Risk of Ruin, P95 DD, CI	Strategy Decision Engine	NO
Walk-Forward	WFE, Robustness, Stability	Strategy Decision Engine	NO
Backtest Engine	Sharpe, PnL, Win Rate, PF	Strategy Decision Engine	NO
Evolution Engine	Improved strategies	Capital Deployment	NO
Capital Allocation	Position sizes, methods	Paper Trading Engine	NO
Strategy State Mgr	State transitions	Execution Engine	NO
Brain Orchestrator	TokenAnalysis, action	Decision Engine (token)	YES

Table 2: Data flow connections — current state

Part II: Quant Developer Perspective

2.1 Strengths in the Codebase

The codebase demonstrates solid quantitative engineering in several key areas. The Monte Carlo Simulator stands out as the highest-quality module: it uses a seeded PRNG (Linear Congruential Generator with period 2^{32}) for reproducibility, implements Fisher-Yates shuffle using the seeded PRNG for unbiased permutation, computes confidence intervals at P5/P25/P50/P75/P95 for equity, drawdown, Sharpe, and win rate, and generates risk-of-ruin and probability-of-profit metrics. The memory-efficient design stores only aggregate statistics per simulation rather than full equity curves, and the human-readable report generator with box-drawing characters is a thoughtful touch for debugging.

The Walk-Forward Engine is similarly well-constructed with both Rolling and Anchored modes, weighted average WFE calculation (weighting by trade count for statistical significance), parameter stability measurement from out-of-sample win rate consistency, and a four-tier robustness assessment (ROBUST/MARGINAL/OVERFIT/INSUFFICIENT_DATA). The Capital Allocation Engine covers an impressive 16 methods including Risk Parity with Spinu 2013 iterative algorithm, Markowitz with Gauss-Jordan matrix inversion, and Kelly Fractional (half-Kelly). These are not toy implementations —

the mathematical formulations are correct and production-ready.

2.2 Critical Problems Identified

Problem 1: Decision Engine is Token-Level, Not Strategy-Level

The existing Decision Engine at `decision-engine.ts` decides whether to operate on an individual TOKEN based on its lifecycle phase and operability score. It produces verdicts like OPERATE/SKIP/WATCH/EXIT. This is a token-level decision — "should I trade this token?" The Strategy Decision Engine we need is a fundamentally different layer that evaluates whether a complete STRATEGY (with its backtest results, Monte Carlo risk profile, Walk-Forward validation, and live track record) deserves capital allocation. These are two distinct decisions, both necessary, but only the first currently exists. The Strategy Decision Engine must produce verdicts like ACTIVE/PAUSED/RETRAIN/REJECT/REDUCE CAPITAL/INCREASE CAPITAL with associated capital recommendations and allocation methods.

Problem 2: Capital Allocation is Disconnected

The Capital Allocation Engine implements 16 methods with correct mathematical formulations, but no module in the system actually calls it in an automated pipeline. The Paper Trading Engine uses its own `calculatePositionSize()` which simply divides available capital by remaining position slots — a naive equal-weight approach. The Backtest Engine uses `allocationMethod` from the TradingSystem configuration but never invokes the CapitalAllocationEngine. The code exists but is orphaned — impressive academically but functionally dead code. This is the most wasteful gap in the project: thousands of lines of sophisticated allocation logic that nobody uses.

Problem 3: Evolution Engine Uses Math.random()

The `mutateParams()` method in StrategyEvolutionEngine uses `Math.random()` for mutation, making the evolution process non-reproducible — each execution produces different results. In quantitative finance, reproducibility is mandatory. The Monte Carlo Simulator correctly uses a seeded PRNG for reproducibility, but the Evolution Engine does not, creating an architectural inconsistency. This must be fixed by replacing Math.random() with the same seeded PRNG pattern used in the Monte Carlo module, ensuring that given the same seed, the evolution produces identical results.

Problem 4: Feedback Loop is Broken

The FeedbackLoopEngine exists but there is no automatic mechanism that feeds paper trading results back to parameter optimization. The system evolves strategies using historical backtests but does not learn from live operations. When a strategy starts underperforming in paper trading, there is no pipeline that detects this degradation and triggers parameter re-optimization or strategy retirement. This creates a dangerous situation: a strategy can pass its backtest with flying colors, fail in live conditions, and nobody (neither human nor system) notices or acts on the divergence. The feedback loop must close: paper trading results must feed back into the evolution engine for continuous strategy refinement.

Problem 5: Cross-Correlation Uses Static Priors

The Cross-Correlation Engine calculates P(outcome | trader_behavior x pattern x phase) using Bayesian combination, but the priors (PHASE_PRIORS, PATTERN_LIKELIHOODS, BEHAVIOR_LIKELIHOODS) are hardcoded constants that never update dynamically. In production, these priors should be recalculated as observations accumulate — a form of Bayesian updating. With static priors, the cross-correlation engine cannot adapt to regime changes or evolving market microstructure. This is a partial implementation that works for initial analysis but fails for continuous operation.

Problem 6: No Real Market Regime Detection

The `inferRegime()` function in the Decision Engine is a simplified heuristic based on lifecycle phase plus volatility. It produces four states (BULL/BEAR/SIDEWAYS/HIGH_VOL) without quantitative grounding. Real regime detection requires analysis of market-wide data: trend strength (ADX, Aroon), volatility clustering (GARCH), correlation structure (PCA on returns), and liquidity conditions. Without proper regime detection, capital allocation cannot adapt dynamically — the system uses the same allocation approach regardless of whether the market is trending, mean-reverting, or experiencing a liquidity crisis.

2.3 Architectural Design Errors

Beyond individual module problems, the architecture has systemic design issues that must be addressed. First, excessive weak coupling: modules are so decoupled they do not communicate. Each is an island. The Brain produces analysis, the Backtest Engine produces results, the Monte Carlo produces simulations, but nobody unifies them. Second, there is no event bus or pipeline orchestrator. Modules are called ad-hoc from individual API routes with no structured data flow guaranteeing that each module receives inputs from previous modules. Third, the DecisionLog Prisma schema is insufficient — it lacks fields for MC risk-of-ruin, WFE, overfitting score, robustness score, validation grade, and capital recommendation. The schema must be extended significantly to support the Strategy Decision Engine.

Part III: Portfolio Manager Perspective

3.1 What Works for Portfolio Management

The TradingSystem structure with five layers (assetFilter, phaseConfig, entrySignal, executionConfig, exitSignal) is well-designed and provides the flexibility needed for diverse strategy configurations. The database persistence of paper trading sessions, positions, and trades is solid, enabling recovery from server restarts and historical analysis. The CompoundGrowthTracker model in the Prisma schema demonstrates that capital evolution tracking was considered during design. The StrategyStateHistory model provides a complete audit trail of state transitions with timestamps, metrics, and evolution data — essential for portfolio governance.

3.2 Critical Problems

Problem 1: No Portfolio Concept Exists

Each strategy operates independently. There is no mental or data model of "my total strategy portfolio." If Strategy A and Strategy B are both LONG on a meme token on SOL, the concentration risk is enormous and nobody detects it. The system has no concept of portfolio-level metrics: no portfolio Sharpe, no portfolio Sortino, no portfolio max drawdown, no rolling correlation between strategies. These metrics are essential for allocation decisions. Without a portfolio model, the system cannot answer fundamental questions like "How much of my capital is exposed to SOL meme tokens?" or "What is my portfolio VaR?" or "Which strategies are highly correlated and should not both be active simultaneously?"

Problem 2: Capital Allocation is Per-Strategy, Not Per-Portfolio

The `calculatePositionSize()` in the Paper Trading Engine simply divides available capital among remaining position slots. It does not consider: correlation between positions, sector concentration, portfolio drawdown, or any of the 16 allocation methods available in the CapitalAllocationEngine. Even Risk Parity, the most basic portfolio-level method, is never applied. The system treats each position as independent when in reality, positions in correlated assets act as a single large bet. For a platform that aspires to manage capital, this is the single most impactful gap to close.

Problem 3: No Kill Switches

If a strategy starts losing catastrophically (common in crypto), there is no way to stop it automatically. The user must be watching the dashboard manually. For a system that aspires to semi-autonomous operation, this is unacceptable. Kill switches must exist at three levels: position-level (individual position hits emergency stop), strategy-level (strategy drawdown exceeds threshold), and portfolio-level (total portfolio drawdown exceeds threshold). The StrategyStateManager records PAUSED states but does not control execution — a PAUSED strategy can still be traded if the Paper Trading Engine does not check its state.

Problem 4: Evolution Does Not Feed Capital Decisions

The Evolution Engine produces improved strategies but there is no mechanism that says "the child strategy outperformed the parent, migrate capital from parent to child." It is optimization without deployment. The system can evolve a strategy from Sharpe 1.2 to 1.8, but capital continues to flow to the old strategy unless manually redirected. This gap means the evolution engine produces intellectual value but not financial value — the improved strategy exists in the database but not in the live portfolio.

Problem 5: Allocation Method Selection is Manual and Arbitrary

The system has 16 allocation methods (Markowitz, Risk Parity, Kelly, etc.) but who decides WHICH method to use and WHEN? The answer should be: the Strategy Decision Engine, based on market regime and portfolio track record. Currently, the choice is manual and arbitrary. In a bull market, Kelly Fractional is appropriate for aggressive growth. In a bear market, Max Drawdown Control is safer. With multiple uncorrelated strategies, Risk Parity optimizes diversification. The method selection must be automated and regime-aware.

Part IV: Risk Manager Perspective

4.1 Strengths in Risk Management

The operability score is a genuine innovation — estimating the impact of fees plus slippage before executing a trade is something most crypto tools do not do. The bot detection engine with 13 bot types is comprehensive and provides valuable intelligence for filtering noisy market signals. The Alert Engine with cooldowns, rules, and webhook delivery is well-designed for operational monitoring. The Data Quality Gate provides a necessary protection layer against garbage-in-garbage-out scenarios. These components form a solid risk management foundation, but they are individual tools rather than an integrated risk framework.

4.2 Critical Problems

Problem 1: No Risk Budget

How much total risk can the portfolio take? What is the daily or weekly Value at Risk? Without an explicit risk budget, any allocation is arbitrary. A risk budget defines the maximum acceptable loss at portfolio, strategy, and position levels. It should specify: maximum portfolio drawdown (e.g., 20%), maximum single-strategy drawdown (e.g., 15%), maximum position concentration (e.g., 30% in single token), maximum sector concentration (e.g., 40% in meme tokens), and maximum chain concentration (e.g., 60% on SOL). Without these constraints, the system can accidentally concentrate all capital in correlated risky positions.

Problem 2: Risk of Ruin Threshold Too Lax

The current veto of Risk of Ruin greater than 5% is too lenient for real capital. In an institutional context, risk of ruin should be less than 1%. The 5% threshold is acceptable only for paper trading. The system should implement risk profiles: Conservative (Risk of Ruin less than 1%), Moderate (less than 3%), and Aggressive (less than 5%). The profile should be selectable per portfolio and enforceable by the Strategy Decision Engine as a hard veto.

Problem 3: No Drawdown Limits

If a strategy loses 30%, it should be paused automatically. If the portfolio loses 20%, everything should pause. This does not exist. The CapitalStrategyManager defines drawdown thresholds (WARNING 10%, DANGER 15%, CRITICAL 25%) but these are passive labels, not active circuit breakers. The system should implement escalating responses: at WARNING, reduce position sizes by 50%; at DANGER, pause new entries; at CRITICAL, close all positions and pause the entire portfolio. These must be automatic, not manual.

Problem 4: Monte Carlo Lacks Block Bootstrap

The current Fisher-Yates shuffle destroys the temporal structure of trades. In financial markets, returns exhibit autocorrelation and volatility clustering — consecutive trades are not independent. Block Bootstrap (with block size approximately equal to the square root of the number of trades) preserves this temporal structure and produces more realistic confidence intervals. Without it, the Monte Carlo results

are optimistic because they assume trade independence, which underestimates the probability of consecutive losses and overestimates the Sharpe ratio confidence interval.

Problem 5: No Stress Testing

What happens if Bitcoin drops 30% in a day? How do all strategies behave simultaneously? Without stress testing of extreme scenarios, the real risk is unknown. The Monte Carlo engine should support stress scenarios: inject artificial shocks (-30%, -50%, -90% portfolio value) at random points in the simulation, simulate flash crashes with correlated position failures, and model rug pulls with instant -95% moves on individual tokens. These stress tests should be mandatory for any strategy seeking ACTIVE status.

Problem 6: No Position-Level Emergency Stop

The paper trading system has trailing stops and stop-loss/take-profit, but there is no emergency kill switch that closes EVERYTHING if portfolio drawdown exceeds a critical threshold. This is the nuclear option that must exist: when the portfolio has lost more than X%, close all positions immediately, pause all strategies, and alert the user. This is not about optimal exit — it is about capital preservation when the system is malfunctioning or market conditions are extreme.

Part V: 30 Key Questions and Answers

5.1 Architecture and Strategy Decision Engine

Q1: Validation Hub vs Strategy Decision Engine — Which direction?

Strategy Decision Engine (SDE). A Validation Hub sounds like a passive control panel — validating and displaying results. A Decision Engine sounds like a motor that PRODUCES decisions. The semantic distinction matters because it guides design. The SDE must receive inputs from all modules (MC, WF, backtest, operability, regime), apply hard vetoes first (any veto = REJECT), then calculate composite scores (robustness, overfitting, stability), and finally produce an actionable DECISION: ACTIVE/PAUSED/RETRAIN/REJECT/REDUCE CAPITAL/INCREASE CAPITAL with position size recommendation and allocation method.

Q2: Where does the SDE live in the architecture?

As a SUPERIOR layer that consumes outputs from all other modules. It replaces nothing. It is an aggregator and decision maker: Backtest Engine, Monte Carlo, Walk-Forward, Operability, Regime Detector, and Evolution Tree all feed into the SDE, which then produces DECISION plus Capital Allocation. No existing module is modified — the SDE sits above them all as the orchestration layer.

Q3: What is the minimum viable output of the SDE?

A StrategyDecision object containing: strategyId, verdict (ACTIVE/PAUSED/RETRAIN/REJECT/REDUCE/INCREASE), validationGrade (A through F),

robustnessScore (0-100), overfittingScore (0-100), stabilityScore (0-100), vetoFailures list, capitalRecommendation (with action, targetPct, method, reason), and nextReviewDate. This interface is the contract between the SDE and all consumers.

Q4: Are the hard vetoes from v1.0 correct?

Mostly yes, with adjustments: Risk of Ruin greater than 5% is too lax — use profiles (Conservative less than 1%, Moderate less than 3%, Aggressive less than 5%). WFE less than 30% is correct as veto. Total Trades less than 30 is correct but consider less than 50 for more statistical significance. Max Drawdown greater than 50% is correct. Win Rate less than 35% should only be a veto if avgLoss/avgWin is greater than 2.5 — a system with 30% win rate and 4:1 payoff ratio is profitable.

Q5: Can the existing Decision Engine be reused?

Yes, but as a component for TOKEN-level decisions. Rename it to token-decision-engine.ts. The new strategy-decision-engine.ts is a different layer operating at the strategy/portfolio level. Both are needed — the token-level engine decides which tokens to trade, the strategy-level engine decides which strategies deserve capital. They operate at different abstraction levels and should not be merged.

Q6: How should the SDE handle conflicting signals?

Vetoes are absolute — any single veto overrides all positive signals. Beyond vetoes, use a weighted scoring system where each module contributes to a composite score: Monte Carlo (30% weight on risk-of-ruin and P95 DD), Walk-Forward (25% weight on WFE and stability), Backtest (25% weight on Sharpe and profit factor), Operability (10% weight), Regime (10% weight). The composite score maps to validation grades: A (80-100), B (65-79), C (50-64), D (35-49), F (0-34). Grade D or below = REJECT.

5.2 Capital Allocation

Q7: Which of the 16 methods should be the default?

Kelly Fractional (half-Kelly) as default for position sizing, and Risk Parity for portfolio allocation. Kelly is the only method that maximizes long-term growth with controlled risk. Risk Parity is superior to equal-weight because it adjusts for volatility. Markowitz is too sensitive to estimation error in crypto markets where return distributions are fat-tailed and non-stationary.

Q8: How should the allocation method be chosen dynamically?

The SDE decides based on market regime: Bull market uses Kelly plus trend following for aggressive positioning. Bear market uses Max Drawdown Control for defensive positioning. High volatility uses Volatility Targeting to reduce exposure. Multiple uncorrelated strategies use Risk Parity for optimal diversification. Single strategy with high confidence uses Kelly Modified for controlled concentration. The method selection must be automated and regime-aware, not manual.

Q9: What about methods that are never used?

Deprecate or remove: FIXED_AMOUNT (does not scale), FIXED_RATIO (too conservative), RL_ALLOCATION (Q-table is a toy without a real simulation environment), META_ALLOCATION (needs more data than available). This leaves 12 useful methods. The removed methods should be archived but not deleted — they may become relevant as the system matures.

Q10: How to connect Capital Allocation to real paper trading?

Replace calculatePositionSize() in PaperTradingEngine with a call to CapitalAllocationEngine. The SDE determines the method, and CapitalAllocationEngine calculates the size. The code already exists — only the connection is missing. This is the highest-ROI integration task in the entire project: approximately 50 lines of integration code unlock 2,000+ lines of allocation logic.

Q11: Should we implement portfolio-level allocation?

Yes, and it is essential. Currently each strategy gets its own capital silo. Portfolio-level allocation considers: cross-strategy correlation (reduce allocation to correlated strategies), sector/chain concentration limits, portfolio drawdown as a constraint, and dynamic rebalancing as strategies change performance. This requires a Portfolio model in the database and a portfolio-level allocation service that sits above the individual strategy allocation.

5.3 Evolution Tree

Q12: Is the Evolution Engine well-designed?

The general structure is sound: genetic algorithm with adaptive mutation rate, early stopping, quality filtering, and composite scoring (Sharpe 30%, PnL 25%, WinRate 20%, PF 10%, DD -10%). However, Math.random() makes it non-reproducible, there is no crossover (only mutation, which limits parameter space exploration), and no complexity penalty (a strategy with 7 mutated parameters should be penalized versus one with 2). Fix Math.random() first, then add crossover, then add complexity penalty.

Q13: How to connect evolution with capital deployment?

When a child strategy outperforms its parent by a margin (e.g., Sharpe improvement greater than 0.3), the SDE should automatically evaluate the child through full validation (MC + WF + operability). If the child passes, capital migrates from parent to child with a gradual transition (e.g., 25% per day over 4 days) to avoid abrupt position changes. The parent is marked as SUPERSEDED in the evolution tree.

Q14: Should evolution be continuous or scheduled?

Both. Continuous micro-evolution (small parameter tweaks) runs daily during low-activity periods. Macro-evolution (structural changes, new strategy variants) runs weekly. The evolution loop should be interruptible — if a strategy is actively trading, do not mutate its parameters mid-session. Wait for the next scheduled review window.

5.4 Monte Carlo Institutional Upgrades

Q15: What specific MC upgrades are needed?

Three critical upgrades: (1) Block Bootstrap with block size = sqrt(n) to preserve temporal autocorrelation. (2) Stress Scenario injection: -30%, -50%, -90% portfolio shocks at random simulation points, plus flash crash (correlated -80% across all positions) and rug pull (-95% on single token). (3) Consecutive Loss Distribution: calculate P(N consecutive losses) for N = 3, 5, 7, 10, which is critical for drawdown estimation and psychological risk assessment.

Q16: Is the current PRNG sufficient?

The LCG with period 2^{32} is adequate for current needs (10,000 simulations x 1,000 trades). For institutional-grade simulations with 100,000+ paths, consider upgrading to Mersenne Twister (period 2^{19937}). This is a low-priority upgrade — the current PRNG is sufficient for the foreseeable future.

Q17: How should MC results feed the SDE?

The MC engine should output a structured MCRiskProfile containing: riskOfRuin, p95MaxDrawdown, probabilityOfProfit, sharpeConfidenceInterval (P5-P95), expectedShortfall (CVaR at 95%), and stressTestResults. The SDE uses riskOfRuin as a hard veto and p95MaxDrawdown + probabilityOfProfit as scoring inputs. Stress test failures add penalty points to the composite score.

5.5 Walk-Forward Professional Upgrades

Q18: What WF upgrades are needed?

Three upgrades: (1) Parameter optimization within windows — currently the same system template is used for both train and test. Real WFA requires optimizing parameters in the training window and testing those optimized parameters out-of-sample. (2) Parameter Drift Analysis — track how optimal parameters change between windows. High drift = low stability = likely overfitting. (3) Overfitting Probability — calculate P(overfit) from the WFE distribution using the methodology from Bailey, Borwein, Lopez de Prado (2014) 'Pseudo-Mathematics and Financial Charlatanism'.

Q19: Is anchored or rolling WFA better for crypto?

Rolling WFA is better for crypto because: market microstructure changes rapidly (new tokens, new DEXs, new MEV strategies), regime shifts are frequent (bull/bear cycles every 6-18 months), and anchored WFA gives too much weight to ancient history. Rolling WFA with a 60/40 train/test split and 6-month training window provides the best balance between statistical significance and relevance.

Q20: How should WF results feed the SDE?

The WF engine should output a WFValidationProfile containing: aggregateWFE, parameterStability, robustnessAssessment, overfittingProbability, and windowResults array. The SDE uses: WFE less than 30% as hard veto, robustnessAssessment OVERFIT as hard veto, parameterStability as scoring input,

and overfittingProbability as scoring modifier.

5.6 Regime Detection, Kill Switches, and Autonomous Operation

Q21: How to implement real regime detection?

Use a multi-factor regime classifier with three components: (1) Trend strength via ADX and Aroon on BTC/ETH daily charts. (2) Volatility regime via realized volatility percentile rank (compare current 30-day vol to 1-year distribution). (3) Correlation structure via PCA on top-20 crypto returns — high first-component loading = risk-on correlated market. Combine the three factors into a 4-state regime: TRENDING, MEAN_REVERTING, HIGH_VOL, CRISIS. Update regime classification daily.

Q22: Kill switch architecture?

Three levels with escalating severity: Position-level (individual position loss exceeds X% or token drops Y% in 1 hour), Strategy-level (strategy drawdown exceeds threshold or Sharpe drops below 0 for 30 days), Portfolio-level (total portfolio drawdown exceeds threshold or daily VaR exceeded by 2x). Each level triggers an automatic response: reduce, pause, or emergency close. All kill switch activations are logged and require manual acknowledgment before re-enabling.

Q23: Risk budget implementation?

Define a RiskBudget model with: maxPortfolioDD (20%), maxStrategyDD (15%), maxPositionConcentration (30%), maxSectorConcentration (40%), maxChainConcentration (60%), maxCorrelatedExposure (50%). The RiskBudget is enforced by the SDE before any capital allocation. Violations are hard blocks, not warnings. The budget should be configurable per risk profile (Conservative/Moderate/Aggressive).

Q24: How to achieve semi-autonomous operation?

Semi-autonomous means the system can operate without constant human supervision but requires human approval for significant decisions. Implementation: the SDE produces decisions automatically, capital allocation is automated, kill switches are automated, but new strategy activation, significant allocation changes (greater than 20%), and kill switch resets require human approval via the dashboard. The system runs autonomously within defined guardrails and escalates decisions that fall outside them.

Q25: Path to fully autonomous operation?

Fully autonomous requires: (1) Strategy Decision Engine with proven track record (6+ months of semi-autonomous operation with positive results). (2) Regime detection with validated accuracy (backtested over multiple cycles). (3) Kill switches that have been tested in production (triggered correctly during real drawdowns). (4) Capital allocation that has been validated through paper trading with real market data. (5) Human oversight reduced to weekly reviews. This is a 12-18 month journey from semi-autonomous to fully autonomous.

Q26: Should we use LLM/AI for decision-making?

LLM should be used as an advisory layer, not a decision maker. The SDE must be deterministic and rule-based — every decision must be auditable and reproducible. LLM can provide: narrative explanations of decisions, regime analysis summaries, and anomaly detection flagging. But the actual verdict (ACTIVE/PAUSED/REJECT) must come from deterministic scoring. LLM-based decisions are untestable, unreproducible, and legally problematic for fiduciary responsibility.

Q27: How to handle strategy correlation in the portfolio?

Implement rolling correlation calculation between strategy equity curves (30-day rolling window). Strategies with correlation greater than 0.7 should not both be at maximum allocation. The SDE should reduce capital to correlated strategies proportionally. Use hierarchical clustering to identify strategy groups and enforce concentration limits per cluster.

Q28: What database schema changes are needed?

Add models: Portfolio (id, name, riskProfile, totalCapital, createdAt), PortfolioAllocation (id, portfolioId, strategyId, targetPct, currentPct, lastRebalanced), RiskBudget (id, portfolioId, maxPortfolioDD, maxStrategyDD, maxConcentration, maxCorrelated), KillSwitch (id, level, threshold, action, triggeredAt, acknowledgedAt, acknowledgedBy), StrategyValidation (id, strategyId, validationGrade, robustnessScore, overfittingScore, stabilityScore, mcRiskProfile, wfValidationProfile, capitalRecommendation, validUntil, reviewedBy). Extend DecisionLog with: mcRiskOfRuin, wfEfficiency, overfittingScore, robustnessScore, validationGrade, capitalRecommendation.

Q29: What is the testing strategy for the SDE?

Three layers: (1) Unit tests for each scoring component with fixed inputs and expected outputs. (2) Integration tests with synthetic strategy data that exercises all veto and scoring paths. (3) Backtest-of-backtests: run the SDE on historical strategy performance data and measure whether its decisions would have improved portfolio returns. The SDE must pass all three layers before going live.

Q30: What is the single most important next step?

Build the Strategy Decision Engine. Everything else is secondary. The SDE is the keystone that transforms this project from an analytics dashboard into a trading system. Without it, the 16 allocation methods are unused, Monte Carlo results are informational but not actionable, Walk-Forward validation is academic but not operational, and the evolution engine produces improvements that are never deployed. The SDE unifies all modules into a decision pipeline and makes the system capable of producing its core output: capital allocation decisions.

Part VI: Strategy Decision Engine Design

6.1 Architecture Overview

The Strategy Decision Engine (SDE) operates as the top-level orchestration layer in the system architecture. It receives structured inputs from all validation modules, applies a deterministic decision pipeline (vetoes, scoring, verdict, capital recommendation), and produces actionable decisions that downstream systems (Paper Trading, Capital Allocation, Evolution) consume. The SDE is stateless with respect to decision logic — given the same inputs, it always produces the same output. State (track record, learning) is persisted to the database and loaded on each evaluation cycle.

6.2 Decision Pipeline

The decision pipeline has four sequential stages. Stage 1: Hard Veto Check — any single veto failure results in immediate REJECT without further evaluation. Stage 2: Composite Scoring — weighted score from all modules. Stage 3: Verdict Determination — score maps to verdict based on thresholds and historical context. Stage 4: Capital Recommendation — verdict plus regime plus portfolio state determines allocation action, target percentage, and allocation method.

Hard Veto Rules

Veto	Condition	Profile: Conservative	Profile: Moderate	Profile: Aggressive
Risk of Ruin	MC simulation	greater than 1%	greater than 3%	greater than 5%
Walk-Forward Efficiency	WFA result	less than 40%	less than 30%	less than 20%
Total Trades	Backtest count	less than 50	less than 30	less than 20
Max Drawdown	Backtest metric	greater than 30%	greater than 40%	greater than 50%
Robustness	WF assessment	OVERFIT	OVERFIT	OVERFIT
Data Quality	Quality gate	UNOPERABLE	UNOPERABLE	RISKY or worse

Table 3: Hard veto rules by risk profile

Composite Scoring Weights

Module	Weight	Key Metrics	Scoring Logic
Monte Carlo	30%	Risk of Ruin, P95 DD, Prob. of Profit	Risk of Ruin mapped to 0-100, P95 DD penalty, PoP bonus
Walk-Forward	25%	WFE, Parameter Stability, Robustness	WFE scaled to 0-100, stability modifier, robustness bonus/penalty
Backtest	25%	Sharpe, Profit Factor, Win Rate, Max DD	Sharpe scaled (0 = 0, 3.0 = 100), PF bonus, DD penalty

Operability	10%	Operability Score, Fee Impact	Score directly used, fee impact penalty
Regime Fit	10%	Strategy-Regime alignment	Trend strategy in trending market = bonus, etc.

Table 4: Composite scoring weights and logic

Verdict Mapping

Score Range	Grade	Verdict	Capital Action
80-100	A	ACTIVE	INCREASE CAPITAL, full allocation
65-79	B	ACTIVE	ALLOCATE, standard allocation
50-64	C	ACTIVE (reduced)	REDUCE CAPITAL, 50% allocation
35-49	D	RETRAIN	HOLD, no new allocation, evolve parameters
0-34	F	REJECT	EXIT, close positions, archive strategy

Table 5: Score-to-verdict mapping

6.3 Core Interface

The StrategyDecision interface defines the contract between the SDE and all consumers. This interface must be stable — downstream systems depend on it. The StrategyValidation database model stores the full decision context for auditability and learning. The SDE is designed for testability: every decision can be reproduced by replaying the same inputs through the pipeline.

Field	Type	Description
strategyId	string	The strategy being evaluated
verdict	enum	ACTIVE / PAUSED / RETRAIN / REJECT / REDUCE / INCREASE
validationGrade	A-F	Composite validation grade
robustnessScore	0-100	Weighted robustness from MC + WF
overfittingScore	0-100	Estimated overfitting probability
stabilityScore	0-100	Parameter stability across windows
vetoFailures	string[]	List of failed veto checks
capitalAction	enum	ALLOCATE / HOLD / REDUCE / EXIT
targetPct	number	Target portfolio percentage
allocationMethod	enum	Recommended allocation method

nextReviewDate	Date	Scheduled re-evaluation date

Table 6: StrategyDecision interface fields

Part VII: Prioritized Roadmap

The roadmap is organized into six sprints, each with a clear deliverable that builds on the previous sprint. The principle is: every sprint must produce a system that is more capable than before. No sprint should leave the system in a broken or regressed state. The highest-priority items are those that close integration gaps, because a connected system is worth more than the sum of its disconnected parts.

Sprint 0: Foundation Fixes (Week 1-2)

Task	Priority	Effort	Impact	Description
Fix Math.random() in Evolution	P0	2h	HIGH	Replace with seeded PRNG matching MC pattern
Connect CapitalAllocation to PaperTrading	P0	4h	CRITICAL	Replace calculatePositionSize() with CapitalAllocationEngine call
Rename decision-engine to token-decision-engine	P1	1h	MEDIUM	Clarify scope: token-level vs strategy-level decisions
Fix StrategyStateManager enforcement	P1	3h	HIGH	PAUSED strategies must not execute trades
Add DecisionLog schema fields	P1	2h	MEDIUM	Add mcRiskOfRuin, wfEfficiency, overfittingScore, validationGrade
Close feedback loop	P1	4h	HIGH	Paper trading results feed back to evolution engine

Table 7: Sprint 0 tasks

Sprint 1: Strategy Decision Engine (Week 3-5)

This is the keystone sprint. The SDE is built from scratch as a new service that consumes outputs from all existing modules. It implements the four-stage decision pipeline (vetoes, scoring, verdict, capital recommendation) described in Part VI. The deliverable is a working SDE that can evaluate any strategy and produce a StrategyDecision with validation grade, verdict, and capital recommendation. The SDE must be fully deterministic and testable.

Task	Effort	Description
SDE Core Pipeline	8h	Veto check, composite scoring, verdict mapping, capital recommendation

SDE API Route	3h	POST /api/strategy-decision/evaluate, GET /api/strategy-decision/history
StrategyValidation DB Model	2h	Prisma model for validation results with full decision context
SDE Unit Tests	4h	Test all veto paths, scoring calculations, verdict mappings
SDE Integration Tests	3h	End-to-end test with real backtest/MC/WF data

Table 8: Sprint 1 tasks

Sprint 2: Monte Carlo Institutional Upgrades (Week 6-7)

Task	Effort	Description
Block Bootstrap	6h	Implement block resampling with block size = sqrt(n), preserving temporal autocorrelation
Stress Scenarios	4h	Inject -30%, -50%, -90% shocks; flash crash; rug pull at random simulation points
Consecutive Loss Distribution	3h	Calculate P(N consecutive losses) for N = 3, 5, 7, 10
MC Risk Profile Output	2h	Structured MCRiskProfile interface for SDE consumption
Update MC Panel UI	4h	Add Block Bootstrap toggle, stress scenario selector, consecutive loss chart

Table 9: Sprint 2 tasks

Sprint 3: Walk-Forward Professional Upgrades + Regime Detection (Week 8-10)

Task	Effort	Description
Parameter Optimization in WF Windows	8h	Optimize parameters in training window, test with those parameters out-of-sample
Parameter Drift Analysis	4h	Track how optimal parameters change between windows, calculate drift score
Overfitting Probability	4h	Implement P(overfit) from WFE distribution per Bailey et al. (2014)
Regime Detection Engine	8h	Multi-factor classifier: ADX + volatility percentile + PCA correlation structure
Regime API + Daily Update	3h	GET /api/market/regime, daily cron job for regime reclassification

Table 10: Sprint 3 tasks

Sprint 4: Kill Switches + Portfolio Monitor (Week 11-13)

Task	Effort	Description
Kill Switch Engine	6h	3-level kill switches: position, strategy, portfolio with configurable thresholds
Risk Budget Model + Enforcement	4h	Portfolio model, risk budget constraints, concentration limits, SDE enforcement
Portfolio VaR Calculation	6h	Historical VaR (95%, 99%) at daily and weekly horizons, CVaR (Expected Shortfall)
Strategy Correlation Monitor	4h	Rolling 30-day correlation between strategy equity curves, clustering, concentration alerts
Emergency Close Pipeline	4h	Portfolio-level emergency position close with alert + logging + acknowledgment
Portfolio Dashboard UI	6h	Portfolio overview, risk budget status, correlation heatmap, kill switch controls

Table 11: Sprint 4 tasks

Sprint 5: Full Integration Pipeline + Autonomous Loop (Week 14-17)

Task	Effort	Description
Automated SDE Evaluation Cycle	4h	Daily cron: evaluate all active strategies through SDE, update verdicts
Capital Allocation Pipeline	6h	SDE verdict drives CapitalAllocationEngine, regime-aware method selection
Evolution-to-Deployment Pipeline	4h	Evolved strategies auto-evaluated by SDE, capital migration on improvement
Human Approval Gate	4h	Approval dashboard for: new strategy activation, allocation changes greater than 20%, kill switch resets
Operational Monitoring	4h	Daily report: active strategies, portfolio risk, recent decisions, pending approvals
End-to-End Integration Test	6h	Full pipeline test: brain scan, strategy creation, backtest, MC, WF, SDE, capital allocation, paper trading

Table 12: Sprint 5 tasks

7.2 Timeline Summary

Sprint	Weeks	Key Deliverable	System Capability After
Sprint 0	1-2	Foundation fixes	Reproducible evolution, connected capital allocation, enforced state management
Sprint 1	3-5	Strategy Decision Engine	Automated strategy evaluation with validation grades and capital recommendations
Sprint 2	6-7	MC institutional upgrades	Block Bootstrap, stress testing, institutional-grade risk profiles
Sprint 3	8-10	WF professional + regime detection	Parameter optimization in WFA, overfitting probability, real regime detection
Sprint 4	11-13	Kill switches + portfolio monitor	Automated circuit breakers, risk budget enforcement, VaR, correlation monitoring
Sprint 5	14-17	Full integration pipeline	Semi-autonomous operation with human approval gates, end-to-end automation

Table 13: Sprint timeline and system capability progression

> **Critical Path:** Sprint 1 (Strategy Decision Engine) is the keystone. Without it, Sprints 2-5 have no consumer for their outputs. Prioritize getting the SDE to a working state before investing in MC/WF upgrades. A simple SDE with basic scoring is more valuable than institutional-grade MC that nobody reads.

Part VIII: Known Bugs and Pending Items

ID	Description	Status	Sprint
B5	saveControlsMutation uses setTimeout(500) fake delay	Open	Sprint 0
B6	smart-money-sync imports analyticsToMetrics from wrong module	Open	Sprint 0
B7	phase-strategy-engine token type incomplete	Open	Sprint 0
B8/B9	strategy-marketplace CATEGORY_META missing bg, stars typed as never[]	Open	Sprint 0
B10	Candles only fetched for top tokens, not user-selected	Partial	Sprint 1
B12	Monte Carlo UI exists but needs Block Bootstrap/stress controls	Partial	Sprint 2
B13	Walk-Forward UI exists but needs parameter optimization viz	Partial	Sprint 3
B14	Chart data source badge missing	Open	Sprint 0

B15-B18	Cleanup, security, minor UI fixes	Open	Sprint 0

Table 14: Known bugs and sprint assignment